

```
void operator delete(void* rawMemory) throw();  
                                     //global 作用域中的正常签名式.  
void operator delete(void* rawMemory, std::size_t size) throw();  
                                     //class 作用域中典型的签名式.
```

因此, 当你只使用正常形式的 new 和 delete, 运行期系统毫无问题可以找出那个“知道如何取消 new 所作所为并恢复旧观”的 delete。然而当你开始声明非正常形式的 operator new, 也就是带有附加参数的 operator new, “究竟哪一个 delete 伴随这个 new”的问题便浮现了。

举个例子, 假设你写了一个 class 专属的 operator new, 要求接受一个 ostream, 用来志记 (logged) 相关分配信息, 同时又写了一个正常形式的 class 专属 operator delete:

```
class Widget {  
public:  
    ...  
    static void* operator new(std::size_t size, std::ostream& logStream)  
        throw(std::bad_alloc);           //非正常形式的 new  
    static void operator delete(void* pMemory std::size_t size)  
        throw();                         //正常的 class 专属 delete  
    ...  
};
```

这个设计有问题, 但在探讨原因之前, 我们需要先绕道, 扼要讨论若干术语。

如果 operator new 接受的参数除了一定会有那个 size_t 之外还有其他, 这便是个所谓的 *placement* new。因此, 上述的 operator new 是个 *placement* 版本。众多 *placement* new 版本中特别有用的一个是“接受一个指针指向对象该被构造之处”, 那样的 operator new 长相如下:

```
void* operator new(std::size_t, void* pMemory) throw();  
                                     //placement new
```

这个版本的 new 已被纳入 C++ 标准程序库, 你只要 #include <new> 就可以取用它。这个 new 的用途之一是负责在 vector 的未使用空间上创建对象。它同时也是最早的 *placement* new 版本。实际上它正是这个函数的命名根据: 一个特定位置上的 new。以上说明意味术语 *placement* new 有多重定义。当人们谈到 *placement* new, 大多数时候他们谈的是此一特定版本, 也就是“唯一额外实参是个 void*”, 少数时候才是指接受任意额外实参之 operator new。上下文语境往往也能够使意义不明晰的含糊话语清晰起来, 但了解这一点相当重要: 一般性术语 “*placement*

new" 意味带任意额外参数的 new, 因为另一个术语 "*placement* delete" 直接派生自它。稍后我们即将遭遇后者。

现在让我们回到 Widget class 的声明式, 也就是先前我说设计有问题的那个。这里的技术困难在于, 那个 class 将引起微妙的内存泄漏。考虑以下客户代码, 它在动态创建一个 Widget 时将相关的分配信息志记 (logs) 于 cerr:

```
Widget* pw = new (std::cerr) Widget; //调用 operator new 并传递 cerr 为其
                                     //ostream 实参; 这个动作会在 Widget
                                     //构造函数抛出异常时泄漏内存
```

再说一次, 如果内存分配成功, 而 Widget 构造函数抛出异常, 运行期系统有责任取消 operator new 的分配并恢复旧观。然而运行期系统无法知道真正被调用的那个 operator new 如何运作, 因此它无法取消分配并恢复旧观, 所以上述做法行不通。取而代之的是, 运行期系统寻找“参数个数和类型都与 operator new 相同”的某个 operator delete。如果找到, 那就是它的调用对象。既然这里的 operator new 接受类型为 ostream& 的额外实参, 所以对应的 operator delete 就应该是:

```
void operator delete(void*, std::ostream&) throw();
```

类似于 new 的 *placement* 版本, operator delete 如果接受额外参数, 便称为 *placement* deletes。现在, 既然 Widget 没有声明 *placement* 版本的 operator delete, 所以运行期系统不知道如何取消并恢复原先对 *placement* new 的调用。于是什也不做。本例之中如果 Widget 构造函数抛出异常, 不会有任何 operator delete 被调用 (那当然不妙)。

规则很简单: 如果一个带额外参数的 operator new 没有“带相同额外参数”的对应版 operator delete, 那么当 new 的内存分配动作需要取消并恢复旧观时就没有任何 operator delete 会被调用。因此, 为了消弭稍早代码中的内存泄漏, Widget 有必要声明一个 *placement* delete, 对应于那个有志记功能 (logging) 的 *placement* new:

```
class Widget {
public:
    ...
    static void* operator new(std::size_t size, std::ostream& logStream)
        throw(std::bad_alloc);
```

```

static void operator delete(void* pMemory) throw();
static void operator delete(void* pMemory, std::ostream& logStream)
    throw();
...
};

```

这样改变之后, 如果以下语句引发 Widget 构造函数抛出异常:

```
Widget* pw = new (std::cerr) Widget;           //一如以往, 但这次不再泄漏
```

对应的 *placement* delete 会被自动调用, 让 Widget 有机会确保不泄漏任何内存。

然而如果没有抛出异常 (通常如此), 而客户代码中有个对应的 delete, 会发生什么事:

```
delete pw;                                     //调用正常的 operator delete
```

就如上一行注释所言, 调用的是正常形式的 operator delete, 而非其 *placement* 版本。 *placement* delete 只有在“伴随 *placement* new 调用而触发的构造函数”出现异常时才会被调用。对着一个指针 (例如上述的 pw) 施行 delete 绝不会导致调用 *placement* delete。不, 绝对不会。

这意味如果要对所有与 *placement* new 相关的内存泄漏宣战, 我们必须同时提供一个正常的 operator delete (用于构造期间无任何异常被抛出) 和一个 *placement* 版本 (用于构造期间有异常被抛出)。后者的额外参数必须和 operator new 一样。只要这样做, 你就再也不会因为难以察觉的内存泄漏而失眠。唔, 至少不是本条款所说的这些难以察觉的内存泄漏。

附带一提, 由于成员函数的名称会掩盖其外围作用域中的相同名称 (见条款 33), 你必须小心避免让 class 专属的 new 掩盖客户期望的其他 new (包括正常版本)。假设你有一个 base class, 其中声明唯一一个 *placement* operator new, 客户端会发现他们无法使用正常形式的 new:

```

class Base {
public:
    ...
    static void* operator new(std::size_t size,
                              std::ostream& logStream)
        throw(std::bad_alloc);           //这个 new 会遮掩正常的 global 形式
    ...
};

```

```
Base* pb = new Base; //错误! 因为正常形式的 operator new 被掩盖.
Base* pb = new (std::cerr) Base; //正确, 调用 Base 的 placement new.
```

同样道理, derived classes 中的 operator news 会掩盖 global 版本和继承而得的 operator new 版本:

```
class Derived: public Base { //继承自先前的 Base
public:
    ...
    static void* operator new(std::size_t size) //重新声明正常形式的
        throw(std::bad_alloc); //形式的 new
    ...
};

Derived* pd = new (std::clog) Derived; //错误! 因为 Base 的
// placement new 被掩盖了.
Derived* pd = new Derived; //没问题, 调用 Derived 的
// operator new.
```

条款 33 更详细地讨论了这种名称遮掩问题。对于撰写内存分配函数, 你需要记住的是, 缺省情况下 C++ 在 global 作用域内提供以下形式的 operator new:

```
void* operator new(std::size_t) throw(std::bad_alloc); //normal new.
void* operator new(std::size_t, void*) throw(); //placement new.
void* operator new(std::size_t,
    const std::nothrow_t&) throw(); //nothrow new,
//见条款 49.
```

如果你在 class 内声明任何 operator news, 它会遮掩上述这些标准形式。除非你的意思就是要阻止 class 的客户使用这些形式, 否则请确保它们在你所生成的任何定制型 operator new 之外还可用。对于每一个可用的 operator new 也请确定提供对应的 operator delete。如果你希望这些函数有着平常的行为, 只要令你的 class 专属版本调用 global 版本即可。

完成以上所言的一个简单做法是, 建立一个 base class, 内含所有正常形式的新和 delete:

```
class StandardNewDeleteForms {
public:
    // normal new/delete
    static void* operator new(std::size_t size) throw(std::bad_alloc)
    { return ::operator new(size); }
    static void operator delete(void* pMemory) throw()
    { ::operator delete(pMemory); }
```

```
// placement new/delete
static void* operator new(std::size_t size, void* ptr) throw()
{ return ::operator new(size, ptr); }

static void operator delete(void* pMemory, void* ptr) throw()
{ return ::operator delete(pMemory, ptr); }

// nothrow new/delete
static void* operator new(std::size_t size, const std::nothrow_t& nt) throw()
{ return ::operator new(size, nt); }

static void operator delete(void *pMemory, const std::nothrow_t&) throw()
{ ::operator delete(pMemory); }
};
```

凡是想以自定形式扩充标准形式的客户, 可利用继承机制及 using 声明式 (见条款 33) 取得标准形式:

```
class Widget: public StandardNewDeleteForms {           //继承标准形式
public:
    using StandardNewDeleteForms::operator new;         //让这些形式可见
    using StandardNewDeleteForms::operator delete;

    static void* operator new(std::size_t size,         //添加一个自定的
                               std::ostream& logStream) //placement new
    { throw(std::bad_alloc); }

    static void operator delete(void* pMemory,          //添加一个对应的
                               std::ostream& logStream) //placement delete
    { throw(); }
    ...
};
```

请记住

- 当你写一个 **placement** operator new, 请确定也写出了对应的 **placement** operator delete。如果没有这样做, 你的程序可能会发生隐微而时断时续的内存泄漏。
- 当你声明 **placement** new 和 **placement** delete, 请确定不要无意识 (非故意) 地遮掩了它们的正常版本。

9

杂项讨论

Miscellany

欢迎来到大杂烩的一章。本章只有 3 个条款，但千万别被低微的数字或不迷人的布景愚弄了，它们都很重要！

第一个条款强调不可以轻忽编译器警告信息。至少，如果你希望你的软件有适当行为的话，别太轻忽它们。第二个条款带你综览 C++ 标准程序库，其中覆盖由 TR1 引进的重大新机能。最后一个条款带你综览 Boost，那是我认为最重要的一个 C++ 泛用型网站。如果你尝试写出高效 C++ 软件，却没有参考这些条款所提供的信息，那么充其量也只是一场事倍功半的恶战。

条款 53：不要轻忽编译器的警告

Pay attention to compiler warnings.

许多程序员习惯性地忽略编译器警告。他们认为，毕竟，如果问题很严重，编译器应该给一个错误信息而非警告信息，不是吗？这种想法对其他语言或许相对无害，但在 C++，我敢打赌编译器作者对于将会发生的事情比你有很好的领悟。举个例子，下面是多多少少都会发生在每个人身上的一个错误：

```
class B {
public:
    virtual void f( ) const;
};

class D: public B {
public:
    virtual void f( );
};
```